

MAT31-5

Applied Probability, Simulation, and Experimental Inference

Monte Carlo, variance reduction, bootstrapping, A/B testing, and likelihood geometry

Estimate a quantity by simulation, attach an honest measure of its uncertainty, and turn the pair into a decision.

Albert School — 22 hours

Preface

In **Probability & Statistics I** (MAT21-2) you learned to build probabilistic models from words: to translate a problem into a sample space, choose the right discrete distribution, and compute expectations and variances. You met the **maximum-likelihood** estimator and watched Bayesian updating shift a posterior as data accumulate. In **Probability & Statistics II** (MAT31-2, running alongside this course) you crossed into the continuous world — densities, the normal distribution, the **Law of Large Numbers** and the **Central Limit Theorem** — and you built the machinery of **hypothesis testing** and **confidence intervals**. From **Descriptive Statistics** (MAT11-5) you carry the habits of disciplined data reasoning: means and variances, covariance and correlation, and a sharp eye for the biases that quietly corrupt a conclusion.

This course puts all of that to work on a single recurring task. A decision-maker asks a numerical question — **what is the expected cost of this policy? how much will this change to the website lift conversions? how confident can we be?** — and the honest answer has two parts: a **number** and a **measure of how much that number might be wrong**. When the question is too tangled for a closed-form formula, we **simulate**: we generate the randomness on a computer, average what we see, and read the uncertainty off the spread. This is the method of **Monte Carlo**, and it is one of the most widely used ideas in all of quantitative work.

Three threads run through the course. The first is **estimation by simulation**: writing a target as an expectation $\mathbb{E}[g(X)]$, approximating it by a sample average, and — crucially — reporting its **standard error** and **confidence interval** rather than a bare number. The second is **making that estimate cheaper and more trustworthy: variance reduction** to sharpen the interval at no extra samples, the **bootstrap** to quantify uncertainty when all you have is a finite dataset and no distribution to simulate from, and **validation** to catch a simulation that is silently wrong. The third is **turning numbers into decisions**: the design and reading of **A/B tests**, the difference between a result that is **statistically** significant and one that actually **matters**, and the **geometry of the likelihood surface** — how the curvature of a two-parameter likelihood tells you how much to trust your estimates.

Keep one slogan in view throughout: **an estimate without its uncertainty is not an answer** — **it is a guess wearing a number's clothes**. Everything here is about producing the uncertainty alongside the estimate, and using both to decide.

Contents

Preface	2
1 Monte Carlo estimation	4
1.1 Everything is an expectation	4
1.2 The sample average approximation	4
1.3 The standard error: how wrong might the average be?	5
1.4 Confidence intervals for the estimate	6
2 Variance reduction	6
2.1 The goal, made precise	7
2.2 Antithetic variates: pair each draw with its mirror image	7
2.3 Control variates: subtract a quantity whose answer you know	8
2.4 Quantifying the impact	9
3 Bootstrapping	9
3.1 The bootstrap idea	9
3.2 Standard errors and confidence intervals from resamples	10
3.3 When the bootstrap fails	10

4	Simulation of stochastic systems	11
4.1	Replications: the unit of simulation	11
5	Validation	12
5.1	Analytical benchmarks	12
5.2	Convergence diagnostics	13
6	Python applications	14
6.1	A reusable Monte Carlo estimator	14
6.2	Antithetic variates	14
6.3	The bootstrap	14
6.4	Validating the output	15
7	A/B testing	15
7.1	Designing the test	15
7.2	How many users do we need?	15
7.3	The multiple-testing problem	16
7.4	When to stop: the peeking problem	17
8	Statistical vs. practical significance	17
9	Likelihood geometry	18
9.1	The likelihood surface	18
9.2	Gradient and Hessian: slope and curvature	18
9.3	Curvature is (inverse) uncertainty	19
9.4	Ridges and local identifiability	19
10	A complete simulation-and-inference workflow	20

1 Monte Carlo estimation

Suppose a logistics team wants the **expected delivery time** of a parcel that passes through three sorting hubs, where the delay at each hub is random and the delays interact through a shared weather variable. Writing down the distribution of the total time, integrating it, and producing a clean formula is hopeless. But **simulating** one parcel's journey is easy: draw the weather, draw the three delays, add them up. Do this ten thousand times and average. That average is a **Monte Carlo estimate** of the expected delivery time — and this chapter makes the idea precise and, just as importantly, equips it with an honest error bar.

1.1 Everything is an expectation

The first conceptual move is to recognise how many different quantities are secretly the **same kind of object**: an expectation.

Definition 1 (Target as an expectation) Let X be a random variable or vector with a known distribution, and g a function. The **target quantity** (or **estimand**) is

$$\mu = \mathbb{E}[g(X)].$$

Worked example — three targets, one form.

- A **mean payoff**: if X is tomorrow's demand and $g(X)$ the resulting profit, the expected profit is $\mathbb{E}[g(X)]$ directly.
- A **probability**: the chance that a project overruns its budget b is

$$\mathbb{P}(C > b) = \mathbb{E}[\mathbb{1}\{C > b\}],$$

the expectation of the **indicator** $g(C) = \mathbb{1}\{C > b\}$, which is 1 when the cost exceeds b and 0 otherwise. A probability is just the average of a 0/1 variable.

- An **integral**: $\int_0^1 e^{-x^2} dx = \mathbb{E}[e^{-U^2}]$ where U is uniform on $[0, 1]$, because averaging a function against the uniform density on $[0, 1]$ **is** integrating it there.

The lesson of the example is worth stating loudly: **probabilities, means, and integrals are all expectations**. Once a quantity is written as $\mathbb{E}[g(X)]$, a single tool estimates it.

1.2 The sample average approximation

The Law of Large Numbers (MAT31-2) says that an average of many independent copies converges to the expectation. Monte Carlo is nothing more than taking that promise literally.

Definition 2 (Sample average approximation (SAA)) Draw independent samples $X_1, \dots, X_n$ from the distribution of X and form

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n g(X_i).$$

By the Law of Large Numbers, $\hat{\mu}_n \rightarrow \mu$ almost surely as $n \rightarrow \infty$.

Worked example — estimating a probability. To estimate $p = \mathbb{P}(C > b)$, simulate $n = 10000$ project costs $C_1, \dots, C_n$, count how many exceed b , and divide by n . If 3174 of them overrun, $\hat{p}_n = 3174/10000 = 0.3174$. Here $g(C_i) = \mathbb{1}\{C_i > b\}$, and the sample average is liter-

ally the **fraction of simulated projects that overran** — exactly the intuitive estimate, now seen as an instance of SAA.

1.3 The standard error: how wrong might the average be?

A single number $\hat{\mu}_n$ is useless without a sense of its reliability. Because $\hat{\mu}_n$ is itself random — a different batch of samples gives a different value — it has a variance, and the square root of that variance is the **standard error**.

Proposition 3 (Unbiasedness and standard error) Let $\sigma^2 = \text{Var}(g(X))$, assumed finite. Then $\hat{\mu}_n$ is **unbiased**, $\mathbb{E}[\hat{\mu}_n] = \mu$, and

$$\text{Var}(\hat{\mu}_n) = \frac{\sigma^2}{n}, \quad \text{SE}(\hat{\mu}_n) = \frac{\sigma}{\sqrt{n}}.$$

In practice σ is unknown and is replaced by the **sample standard deviation**

$$\hat{\sigma} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (g(X_i) - \hat{\mu}_n)^2}, \quad \text{so} \quad \widehat{\text{SE}} = \frac{\hat{\sigma}}{\sqrt{n}}.$$

Proof. Unbiasedness is linearity of expectation: $\mathbb{E}[\hat{\mu}_n] = \frac{1}{n} \sum \mathbb{E}[g(X_i)] = \mu$. For the variance, the $g(X_i)$ are independent with common variance σ^2 , so $\text{Var}(\hat{\mu}_n) = \frac{1}{n^2} \sum \text{Var}(g(X_i)) = \frac{1}{n^2} \cdot n\sigma^2 = \sigma^2/n$ (variance of a sum of independent variables, MAT21-2). The square root is the standard error. $\square$

The single most important consequence is the **rate**. The error shrinks like $1/\sqrt{n}$, not $1/n$:

Common pitfall (The square-root law of Monte Carlo) To **halve** the standard error you need **four times** as many samples; for one more decimal digit of accuracy ($10 \times$ smaller error) you need $100 \times$ the samples. Monte Carlo is robust and general, but it converges **slowly**. This is why the variance-reduction techniques of the next chapter matter so much: they buy accuracy that brute-force sampling would make very expensive.

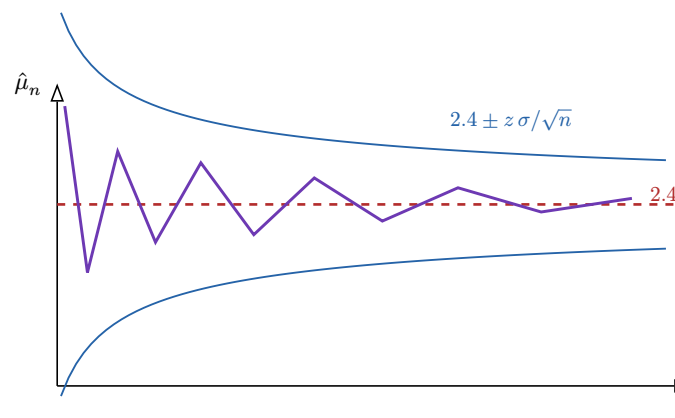


Figure 1: A running Monte Carlo estimate $\hat{\mu}_n$ (purple) as the sample size n grows. It wanders toward the true value μ (red dashed). The blue confidence band has half-width proportional to $1/\sqrt{n}$ — it narrows, but ever more slowly: quartering the width costs sixteen times the samples.

1.4 Confidence intervals for the estimate

The Central Limit Theorem (MAT31-2) tells us more than the rate: it tells us the **shape** of the estimation error. For large n , $\hat{\mu}_n$ is approximately normal around μ , which lets us wrap it in a confidence interval.

Definition 4 (Monte Carlo confidence interval) For large n , an approximate $(1 - \alpha)$ confidence interval for μ is

$$\hat{\mu}_n \pm z_{1-\alpha/2} \frac{\hat{\sigma}}{\sqrt{n}},$$

where $z_{1-\alpha/2}$ is the standard-normal quantile (1.96 for 95%, 2.576 for 99%). Its half-width — the **margin of error** — is $z_{1-\alpha/2} \widehat{\text{SE}}$.

Worked example — reporting an estimate properly. Continuing the overrun example with $\hat{p}_n = 0.3174$ from $n = 10000$: for an indicator, $\hat{\sigma}^2 \approx \hat{p}(1 - \hat{p}) = 0.3174 \cdot 0.6826 = 0.2167$, so $\widehat{\text{SE}} = \sqrt{0.2167/10000} = 0.00466$. The 95% interval is

$$0.3174 \pm 1.96 \cdot 0.00466 = 0.3174 \pm 0.0091 = [0.308, 0.327].$$

The honest report is not “the overrun probability is 0.317” but “we estimate 0.317 with a 95% interval of [0.308, 0.327].” The interval is the answer.

Common pitfall The confidence interval here quantifies **Monte Carlo error** — the uncertainty coming from using **finitely many simulated samples** instead of infinitely many. It says how much $\hat{\mu}_n$ might differ from the true μ of the model you simulated. It says **nothing** about whether that model is a faithful picture of reality: a beautifully tight interval around the wrong model is still wrong. Model error is addressed by **validation** (Chapter 5), not by more samples.

Exercises

1. Write each as an expectation $\mathbb{E}[g(X)]$ and name g : (a) the probability a standard normal exceeds 1.5; (b) the expected absolute value $\mathbb{E}[|Z|]$ of a standard normal; (c) the integral $\int_0^2 \sin(x) dx$ using a uniform X on $[0, 2]$.
2. A Monte Carlo run of $n = 400$ gives $\hat{\mu}_n = 5.2$ and $\hat{\sigma} = 3.0$. Compute the standard error and the 95% confidence interval.
3. How many samples are needed to shrink the margin of error in the previous exercise from its current value to 0.1? Use the square-root law.
4. Explain why estimating a very small probability $p \approx 10^{-4}$ by plain Monte Carlo is expensive: compute the relative standard error SE/p for an indicator and show it blows up as $p \rightarrow 0$.

2 Variance reduction

The square-root law is unforgiving: brute force buys accuracy slowly. The escape is not to take more samples but to take **smarter** ones. A **variance-reduction** technique replaces the naive estimator with a **different** unbiased estimator of the **same** μ that has **smaller variance** —

so the confidence interval shrinks at no extra sampling cost. We study the two the OP names, **antithetic variates** and **control variates**, and learn to **quantify** the gain each delivers.

2.1 The goal, made precise

Recall the interval half-width is $z_{1-\alpha/2} \sigma / \sqrt{n}$. Cutting the **variance** of the estimator by a factor r cuts the **width** by $\sqrt{r}$ — or, equivalently, achieves the same width with r times fewer samples. The whole game is to lower variance without introducing bias.

2.2 Antithetic variates: pair each draw with its mirror image

The intuition is simple and physical. If one simulated scenario comes out **unusually high** by luck, we would like a **paired** scenario that comes out **unusually low**, so that their average cancels the luck. We engineer exactly such pairs.

Definition 5 (Antithetic variates) Many samples are generated as $X = F^{-1}(U)$ from a uniform U on $[0, 1]$ (the **inverse-transform method**, MAT31-2). The **antithetic partner** uses $1 - U$ in place of U : where U is small, $1 - U$ is large. With $n/2$ independent uniforms U_i , form the pairs $X_i = F^{-1}(U_i)$, $X'_i = F^{-1}(1 - U_i)$ and set

$$\hat{\mu}^{\text{anti}} = \frac{1}{n/2} \sum_{i=1}^{n/2} \frac{g(X_i) + g(X'_i)}{2}.$$

Proposition 6 (Variance of the antithetic estimator) Writing $\rho = \text{Corr}(g(X), g(X'))$ for the correlation within a pair,

$$\text{Var}(\hat{\mu}^{\text{anti}}) = \frac{\sigma^2}{n} (1 + \rho).$$

A **negative** correlation $\rho < 0$ makes this **smaller** than the plain σ^2/n ; the more negative, the larger the saving.

Proof. Each pair contributes the average $A_i = (g(X_i) + g(X'_i))/2$, and the pairs are independent. Now $\text{Var}(A_i) = \frac{1}{4}(\sigma^2 + \sigma^2 + 2\rho\sigma^2) = (\sigma^2/2)(1 + \rho)$. Averaging $n/2$ independent such A_i divides by $n/2$: $\text{Var}(\hat{\mu}^{\text{anti}}) = (2/n)(\sigma^2/2)(1 + \rho) = (\sigma^2/n)(1 + \rho)$. $\square$

Worked example — antithetic for a monotone payoff. Estimate $\mathbb{E}[e^U]$ with U uniform on $[0, 1]$ (true value $e - 1 \approx 1.718$). The map $u \mapsto e^u$ is **increasing**, so e^U and e^{1-U} move in **opposite** directions: a large U gives a large e^U but a small e^{1-U} . Their correlation is strongly negative ($\rho \approx -0.97$), so the antithetic estimator has variance roughly $(1 - 0.97) = 0.03$ times the naive one — a thirty-fold reduction, equivalent to using about $30 \times$ more samples for free.

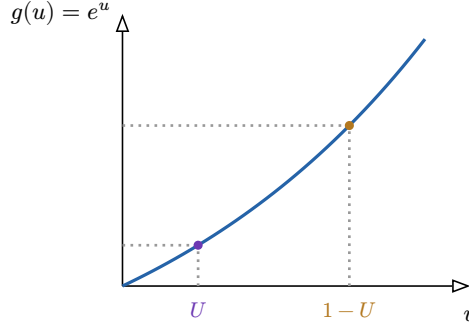


Figure 2: Antithetic variates on an increasing payoff $g(u) = e^u$. A low draw U lands low on the curve; its partner $1 - U$ lands high. Averaging the two cancels much of the sampling luck — the estimator’s variance collapses. The trick **fails** (can even hurt) when g is not monotone, since then $g(U)$ and $g(1 - U)$ need not be negatively correlated.

Common pitfall Antithetic variates help only when $g \circ F^{-1}$ is **monotone** in U , so that the partner is genuinely anti-correlated. For a **symmetric, non-monotone** g (say $g(u)$ symmetric about $u = 1/2$) the pair can be **positively** correlated, and then $1 + \rho > 1$ — the technique **increases** variance. Always check monotonicity, or estimate ρ from a pilot run, before trusting it.

2.3 Control variates: subtract a quantity whose answer you know

The second technique exploits a **side quantity** that travels with $g(X)$ and whose mean is **known exactly**. If our simulated $g(X)$ overshoots on a run where the side quantity also overshoots, we can subtract off the known part of that error.

Definition 7 (Control variates) Let C be a variable computed alongside $g(X)$ on each run, with **known** mean $\mathbb{E}[C] = c$. For a constant b , the **control-variate estimator** is

$$\hat{\mu}^{\text{cv}} = \frac{1}{n} \sum_{i=1}^n (g(X_i) - b(C_i - c)).$$

Because $\mathbb{E}[C_i - c] = 0$, this is **unbiased** for μ for **every** b .

Proposition 8 (Optimal coefficient and variance) The variance $\text{Var}(g(X) - b(C - c))$ is minimised at

$$b^* = \frac{\text{Cov}(g(X), C)}{\text{Var}(C)}, \quad \text{giving} \quad \text{Var}(\hat{\mu}^{\text{cv}}) = \frac{\sigma^2}{n} (1 - \rho_{g,C}^2),$$

where $\rho_{g,C} = \text{Corr}(g(X), C)$. The stronger the correlation — of **either** sign — the larger the reduction; a perfect correlation would drive the variance to zero.

Proof. For a single run, $\text{Var}(g - b(C - c)) = \sigma^2 - 2b \text{Cov}(g, C) + b^2 \text{Var}(C)$, a quadratic in b minimised at $b^* = \text{Cov}(g, C)/\text{Var}(C)$. Substituting and using $\rho_{g,C}^2 = \text{Cov}(g, C)^2/(\sigma^2 \text{Var}(C))$ gives $\sigma^2(1 - \rho_{g,C}^2)$ per run; dividing by n gives the stated variance. In practice b^* is estimated from the same samples (a regression slope of g on C), at a negligible cost in the large- n approximation. $\square$

Worked example — control variate in option-style pricing. Suppose $g(X)$ is a complicated payoff whose mean we want, and $C = X$ is the underlying input with known mean $c = \mathbb{E}[X]$. If the payoff rises with X , then $g(X)$ and C are strongly positively correlated, say $\rho_{g,C} = 0.9$. Then the control-variate variance is $(1 - 0.81) = 0.19$ times the naive one — a factor 5 reduction, the same accuracy as $5 \times$ the samples.

2.4 Quantifying the impact

The OP asks not merely to **apply** a strategy but to **quantify** its effect. The recipe is mechanical and always the same.

To quantify a variance-reduction strategy:

1. Run plain Monte Carlo; record $\widehat{SE}_{\text{naive}}$ and the interval width $w_{\text{naive}} = 2z\widehat{SE}_{\text{naive}}$.
2. Run the reduced estimator on the **same budget**; record $\widehat{SE}_{\text{red}}$ and w_{red} .
3. Report the **variance ratio** $\widehat{SE}_{\text{red}}^2/\widehat{SE}_{\text{naive}}^2$ — which estimates $1 + \rho$ (antithetic) or $1 - \rho_{g,C}^2$ (control variates) — and the **width ratio** $w_{\text{red}}/w_{\text{naive}} = \widehat{SE}_{\text{red}}/\widehat{SE}_{\text{naive}}$, the square root of the variance ratio.

Worked example — a quantified comparison. On the $\mathbb{E}[e^U]$ problem with $n = 10000$: plain Monte Carlo gives $\widehat{SE}_{\text{naive}} = 0.00493$; antithetic gives $\widehat{SE}_{\text{anti}} = 0.00087$. The variance ratio is $(0.00087/0.00493)^2 = 0.031$ — consistent with $1 + \rho$ for $\rho \approx -0.97$ — and the interval is $0.00087/0.00493 \approx 0.18$ times as wide: a roughly 5.7-fold **narrower** interval for the same number of evaluations.

Exercises

5. For the antithetic estimator, at what value of ρ does the technique give **no** benefit? At what value does it **double** the variance?
6. A control variate has $\rho_{g,C} = 0.6$. By what factor does it cut the variance? By what factor does it narrow the confidence interval?
7. Explain why the **sign** of $\rho_{g,C}$ does not matter for control variates but the sign of ρ is everything for antithetic variates.
8. You estimate $\mathbb{E}[g(X)]$ where $g(x) = (x - 1/2)^2$ and X is uniform on $[0, 1]$. Is $g \circ F^{-1}$ monotone in U ? Predict whether antithetic variates will help or hurt, and explain.

3 Bootstrapping

Monte Carlo presupposes a **known distribution** to simulate from. But the everyday situation in data work is the opposite: you hold a **single finite dataset** $x_1, \dots, x_n$ drawn from an **unknown** distribution, you have computed some statistic from it — a median, a correlation, a 90th percentile — and you need a **standard error and confidence interval** for that statistic. There is no formula, and nothing obvious to simulate. The **bootstrap** is the beautiful, almost impudent, resolution.

3.1 The bootstrap idea

If we cannot sample from the true population, we sample from the **best stand-in we have**: the dataset itself. Treating the observed sample as if it were the whole population, we draw new “datasets” from it and watch how the statistic varies.

Definition 9 (The bootstrap) Let $\hat{\theta} = T(x_1, \dots, x_n)$ be a statistic. For $b = 1, \dots, B$:

1. draw a **bootstrap resample** $x_1^*, \dots, x_n^*$ by sampling n values **with replacement** from the original data (so some originals appear several times, others not at all);
2. compute $\hat{\theta}_b^* = T(x_1^*, \dots, x_n^*)$.

The spread of $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$ approximates the **sampling distribution** of $\hat{\theta}$ — the variability you would see across fresh datasets from the true population.

Remark Sampling **with replacement** is essential and is what makes resamples differ. If you sampled **without** replacement you would just recover the original data every time and learn nothing. With replacement, each resample is a slightly reshuffled, reweighted version of the data — a plausible alternative dataset the world might have produced.

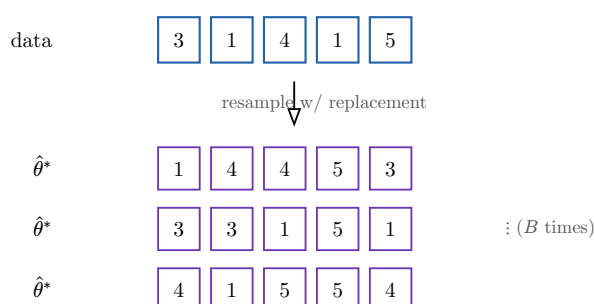


Figure 3: The bootstrap. From the single observed dataset (blue) we draw many resamples of the same size **with replacement** (purple) — repeats and omissions are the point. Recomputing the statistic on each resample traces out its sampling distribution, from which standard errors and confidence intervals follow.

3.2 Standard errors and confidence intervals from resamples

Definition 10 (Bootstrap standard error and percentile interval) The **bootstrap standard error** of $\hat{\theta}$ is the sample standard deviation of the resampled values $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$. The **percentile** $(1 - \alpha)$ confidence interval is the interval between the $\alpha/2$ and $1 - \alpha/2$ empirical quantiles of those values.

Worked example — a confidence interval for a median. There is no tidy closed-form standard error for a sample median, but the bootstrap supplies one effortlessly. Given $n = 200$ incomes, resample $B = 10000$ times, take the median of each resample, and read off the 2.5th and 97.5th percentiles of the 10000 medians: that interval is a 95% confidence interval for the population median. The same three lines of code would have produced an interval for a trimmed mean, an inter-quartile range, or a correlation — **the bootstrap does not care what the statistic is**.

3.3 When the bootstrap fails

The bootstrap can feel like magic, and like all magic it has limits the OP insists you can name. The principle behind every failure is the same: **the resample can only contain information the original sample already holds**.

Common pitfall (Failure modes of the bootstrap)

- **Small samples.** With n tiny, the empirical distribution is a poor stand-in for the population, and resampling from 5 points cannot manufacture the diversity of the true distribution. Bootstrap intervals are then unreliable and often **too narrow**.
- **Heavy tails / infinite variance.** When extreme values dominate (e.g. a Cauchy-like or power-law distribution), the statistic is driven by rare giant observations the sample under-represents, and the resampling distribution does not track the true sampling distribution.
- **Statistics of extremes.** For the sample **maximum** (or minimum), the bootstrap fails outright: the resampled maximum can never **exceed** the observed maximum, so its distribution is degenerate and bunched against a wall — nothing like the true distribution of the maximum.

Worked example — why the maximum breaks it. Let $\hat{\theta} = \max(x_1, \dots, x_n)$. Every resample is built from the original values, so $\hat{\theta}^* \leq \max_i x_i$ **always**, with $\hat{\theta}^*$ equal to the true maximum whenever that value is drawn at least once (probability $1 - (1 - 1/n)^n \approx 1 - e^{-1} \approx 0.63$). The bootstrap distribution thus piles 63% of its mass on a single point and never goes above it — a useless picture of how the maximum actually varies. This is the canonical “bootstrap fails” example to keep in mind.

Exercises

9. Describe in words how you would bootstrap a 95% confidence interval for the Pearson correlation between two columns of a dataset of $n = 150$ paired observations.
10. Explain why the bootstrap standard error of a statistic tends to be **too small** when n is very small.
11. For $n = 10$, what is the probability that a given original observation appears in a particular resample **at least once**? (Use $1 - (1 - 1/n)^n$.)
12. Give one statistic for which the bootstrap works well and one for which it fails, and justify each in a sentence.

4 Simulation of stochastic systems

Monte Carlo estimates a **single** expectation. Many real questions instead concern a **system that evolves with randomness over time**: a checkout queue, a warehouse’s inventory, an insurer’s yearly claims, a call centre’s staffing. These rarely admit closed-form analysis, but they are straightforward to **simulate** — and the machinery of estimates-with-error-bars carries over unchanged.

4.1 Replications: the unit of simulation

Definition 11 (Simulation workflow)

1. **Specify the model.** List the state variables, the random inputs and their distributions, and the rules that advance the system from one step to the next.
2. **Run one replication.** Generate the random inputs and play the system forward once, recording the **outcome of interest** Y (total wait, end-of-year inventory cost, peak queue length, ...).
3. **Replicate.** Repeat for n **independent** replications to obtain $Y_1, \dots, Y_n$.

4. **Summarise.** Report the mean outcome $\bar{Y}$ with a confidence interval

$$\bar{Y} \pm z_{1-\alpha/2} \hat{\sigma}_Y / \sqrt{n},$$

exactly the Monte Carlo interval of Chapter 1, now applied to whole-system outcomes.

The conceptual point is that a **replication is one sample** of the random variable “system outcome”, and $\bar{Y}$ is a sample-average approximation of $\mathbb{E}[Y]$. Everything from Chapter 1 — unbiasedness, the $1/\sqrt{n}$ rate, the CLT interval — applies verbatim, with Y playing the role of $g(X)$.

Worked example — a single-server queue. Customers arrive at random times and a single server takes a random time to serve each. We want the **expected average wait per customer** over a busy day. There is a closed-form answer only in idealised cases; in general we **simulate**. One replication: generate the day’s arrival and service times, step through the day tracking the queue, and record the average wait Y . Run $n = 1000$ independent days and report $\bar{Y} \pm 1.96 \hat{\sigma}_Y / \sqrt{1000}$. Want to compare two staffing policies? Simulate both and compare their intervals.

Common pitfall Within **one** replication the recorded quantities (successive customers’ waits) are **correlated** — a long queue makes everyone behind wait. So you must not treat the individual waits inside a day as independent samples for the confidence interval. The independent unit is the **whole replication**: build the interval from the n **replication-level** outcomes $Y_1, \dots, Y_n$, not from the many correlated events inside a single run. Mixing these levels is the classic simulation blunder.

Exercises

13. An insurer simulates yearly total claims. One replication yields the total for one simulated year. Across $n = 500$ years it finds $\bar{Y} = 1.20\text{M}$ with $\hat{\sigma}_Y = 0.40\text{M}$. Give the 95% confidence interval for expected annual claims.

14. Why is the **replication**, rather than the individual event within a run, the correct independent unit for a confidence interval?

15. Describe the state variables, random inputs, and outcome of interest you would use to simulate a shop’s daily inventory cost under a “reorder when stock drops below s ” policy.

5 Validation

A simulation that runs without error can still be **wrong** — a mis-specified distribution, a bug in the update rule, an off-by-one in the bookkeeping. Before any number is trusted, it must be **validated**. There are two complementary strategies, and which one you can use depends on whether a closed-form answer exists somewhere to lean on.

5.1 Analytical benchmarks

Definition 12 (Validation against an analytical benchmark) Find a **special case** of your model that **does** have a closed-form answer, run the simulation on that case, and check that the estimate agrees with the known value **within its confidence interval**. If

the known value sits comfortably inside the interval, the machinery is behaving; if it sits far outside, something is wrong.

Worked example — benchmarking a queue simulator. Before trusting a complex queue simulation, set the arrival and service rates to a simple regime (e.g. an M/M/1 queue) where the long-run average wait has a textbook formula. Simulate that case; the formula’s value should land inside your confidence interval. Only once the simulator reproduces the **known** case do you turn it loose on the **unknown** case it was built for. A simulator that fails the benchmark cannot be trusted on the real question.

Common pitfall A successful benchmark is **necessary, not sufficient**. Agreement on one special case does not prove correctness on all cases — a bug may hide in machinery the benchmark does not exercise. Benchmark **several** special cases when you can, and treat validation as accumulating confidence, never as a single pass/fail gate.

5.2 Convergence diagnostics

When no closed-form benchmark exists, validate the **behaviour** of the estimate itself as the sample grows.

Definition 13 (Convergence diagnostics) Plot the **running estimate** $\hat{\mu}_n$ against n and check that it **stabilises** rather than drifting; plot the estimated standard error and check that it shrinks like $1/\sqrt{n}$. A running estimate that keeps drifting, or a standard error that refuses to fall at the right rate, signals a problem — too few samples, a heavy tail violating the CLT’s assumptions, or a bug.

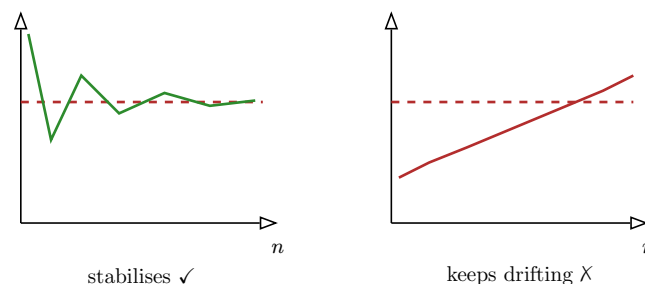


Figure 4: Convergence diagnostics. **Left:** a healthy run — the running estimate settles onto a value and the fluctuations shrink. **Right:** a warning sign — the estimate is still drifting at the largest n , so it has **not** converged; the reported number would be premature. When no analytical benchmark exists, this visual check is the front-line validation tool.

Exercises

16. You simulate a quantity whose true value you do not know. Sketch what the running-estimate plot should look like if the simulation is trustworthy, and what it looks like if you have run too few replications.
17. Your simulator is supposed to estimate $\mathbb{E}[X]$ for an exponential input with rate $\lambda = 2$, whose true mean is $1/2$. Your 95% interval comes out $[0.71, 0.79]$. What do you conclude, and what is the natural next step?

18. Explain why passing a single analytical benchmark does not guarantee the simulator is correct on the case you actually care about.

6 Python applications

The workflows of the previous chapters are short to implement. The point of this chapter is to see how **little** code each idea needs once the concepts are clear — and to insist that every estimate is reported **with its standard error and confidence interval**, never alone.

6.1 A reusable Monte Carlo estimator

```
import numpy as np

def monte_carlo(g, sampler, n, alpha=0.05):
    """Estimate E[g(X)] with its standard error and (1-alpha) CI."""
    x = sampler(n)          # n samples of X
    y = g(x)                # g(X), an array of length n
    mu = y.mean()
    se = y.std(ddof=1) / np.sqrt(n)  # standard error
    z = 1.959963985         # standard-normal 0.975 quantile
    return mu, se, (mu - z*se, mu + z*se)
```

The same function serves every example in Chapter 1: pass a **sampler** that draws X and a function **g**. For replicated **system** simulation (Chapter 4), let **g** play one replication and return its outcome Y — the estimator and interval are identical.

6.2 Antithetic variates

```
def monte_carlo_antithetic(g, inv_cdf, n, alpha=0.05):
    """E[g(X)] via antithetic variates; X = inv_cdf(U)."""
    u = np.random.random(n // 2)
    paired = (g(inv_cdf(u)) + g(inv_cdf(1 - u))) / 2  # n//2 pair-averages
    mu = paired.mean()
    se = paired.std(ddof=1) / np.sqrt(len(paired))
    z = 1.959963985
    return mu, se, (mu - z*se, mu + z*se)
```

To **quantify** the gain, run `monte_carlo` and `monte_carlo_antithetic` on the same budget and compare their `se`: the squared ratio estimates $1 + \rho$.

6.3 The bootstrap

```
def bootstrap_ci(data, stat, B=10000, alpha=0.05):
    """Percentile bootstrap CI for an arbitrary statistic `stat`."""
    n = len(data)
    boot = np.array([stat(np.random.choice(data, n, replace=True))
                     for _ in range(B)])
    se = boot.std(ddof=1)
    lo, hi = np.quantile(boot, [alpha/2, 1 - alpha/2])
    return se, (lo, hi)
```

Passing `stat=np.median`, `stat=np.std`, or any custom function reuses the very same loop — the bootstrap's indifference to the statistic, made concrete.

6.4 Validating the output

```
# Benchmark: estimate E[U] for U ~ Uniform(0,1); known answer is 0.5
mu, se, ci = monte_carlo(lambda u: u, np.random.random, n=100_000)
assert ci[0] <= 0.5 <= ci[1], "benchmark failed: investigate the simulator"
```

Whatever the workflow, the closing step is the same: check the estimate against an **analytical benchmark** or a **convergence plot** before reporting it.

Exercises

19. Use `monte_carlo` to estimate $\int_0^1 e^{-x^2} dx$ by sampling U uniform on $[0, 1]$ and taking $g(u) = e^{-u^2}$. What is the role of each returned value?
20. Modify `bootstrap_ci` to also return the bootstrap **bias** estimate $\bar{\hat{\theta}}^* - \hat{\theta}$, where $\hat{\theta}$ is the statistic on the original data.
21. Explain what the `assert` in the validation snippet checks, and what its failure would tell you.

7 A/B testing

We now turn from **estimating** a quantity to **deciding between two options**. An **A/B test** is a controlled experiment: randomly split users into a **control** group (A, the current version) and a **treatment** group (B, the proposed change), expose each to its version, and compare an outcome — conversion rate, revenue per user, time on page. Randomisation is what licenses a **causal** reading: because assignment is random, a difference in outcomes is attributable to the change itself, not to a hidden difference between the groups (recall the bias warnings of MAT11-5). This chapter is the disciplined design and reading of such a test.

7.1 Designing the test

Definition 14 (A/B test design) Fix, before collecting data:

- the **hypotheses**, e.g. $H_0 : p_A = p_B$ (no effect) versus $H_1 : p_A \neq p_B$ (some effect);
- the **significance level** α (the tolerated Type I error rate, MAT31-2), typically 0.05;
- the **power** $1 - \beta$ (the chance of detecting a real effect), typically 0.8;
- the **minimum detectable effect** δ — the smallest difference deemed **worth detecting** — chosen on business grounds, not statistical ones.

7.2 How many users do we need?

The four design choices are not independent: pin down three and the fourth — the **sample size** — is determined. This is **sample-size logic**, and doing it **before** the experiment is the single most important discipline in A/B testing.

Proposition 15 (Sample size for a two-proportion test) To detect a difference δ in conversion rates at level α with power $1 - \beta$, the required **per-group** sample size is approximately

$$n \approx \left(z_{1-\alpha/2} + z_{1-\beta} \right)^2 \frac{2\bar{p}(1-\bar{p})}{\delta^2},$$

where $\bar{p}$ is the assumed baseline conversion rate.

Worked example — sizing a checkout experiment. A checkout currently converts at $\bar{p} = 0.20$, and the team cares about any lift of at least $\delta = 0.02$ (i.e. to 0.22). With $\alpha = 0.05$ ($z = 1.96$) and power 0.8 ($z = 0.84$),

$$n \approx (1.96 + 0.84)^2 \cdot \frac{2 \cdot 0.20 \cdot 0.80}{0.02^2} = 7.84 \cdot \frac{0.32}{0.0004} \approx 6272$$

users **per group**. Halving the effect we wish to catch to $\delta = 0.01$ would **quadruple** this to about 25000 per group — smaller effects are dramatically more expensive to detect, because n scales like $1/\delta^2$.

Common pitfall Read the dependence carefully: a **smaller** δ , a **smaller** α , or a **larger** power all **increase** n . The common mistake is to launch a test without this calculation, run it until the result “looks significant”, and stop — a procedure that, as we see below, is statistically broken. Compute n first; commit to it.

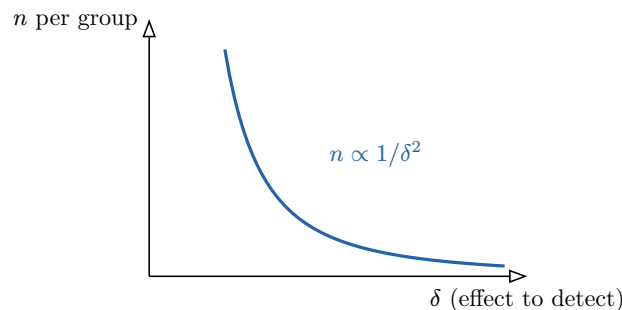


Figure 5: Sample size explodes for small effects. Because $n \propto 1/\delta^2$, halving the minimum detectable effect quadruples the users required. Deciding **how small an effect is worth detecting** is therefore the lever that controls the cost of the whole experiment — a business decision dressed as a statistical one.

7.3 The multiple-testing problem

Run **one** test at level $\alpha = 0.05$ and a true null is wrongly rejected 5% of the time. Run **many** tests — several metrics, several segments, several variants — and false positives accumulate fast.

Common pitfall (Multiple testing inflates false positives) If m independent tests all have **true** null hypotheses, the probability that **at least one** is rejected at level α is

$$1 - (1 - \alpha)^m,$$

which for $\alpha = 0.05$ and $m = 20$ is $1 - 0.95^{20} \approx 0.64$ — a **nearly two-in-three** chance of a spurious “discovery”. The **Bonferroni** correction restores control: test each hypothesis at level α/m , so the overall false-positive rate stays at most α . The price is reduced power, so do not multiply tests gratuitously.

Worked example — the lure of slicing. A team finds no overall effect, then breaks results down by country, device, and age band — twenty slices — and triumphantly reports that the effect is “significant for tablet users in Spain”. With twenty slices, finding **one** spurious significant result is more likely than not. Either pre-register the slices and Bonferroni-correct, or treat such findings as **hypotheses to test afresh**, never as conclusions.

7.4 When to stop: the peeking problem

Common pitfall (Optional stopping (peeking) inflates Type I error) Checking the test repeatedly as data trickle in and stopping **the moment** it crosses significance is **peeking**, and it badly inflates the Type I error rate: given enough looks, a true null will cross the 0.05 line by chance almost surely. The fixes are to **fix the sample size in advance** (run to the pre-computed n , then look **once**), or to use a method **designed** for continuous monitoring (sequential testing / always-valid intervals with spending functions). Naive peeking is not one of the options.

Exercises

22. Recompute the required per-group n for the checkout example if the team raises the power target to 0.9 ($z_{1-\beta} = 1.282$), keeping $\delta = 0.02$.
23. With $\alpha = 0.05$, how many independent true-null tests m make the chance of at least one false positive exceed 0.5?
24. Under Bonferroni correction with $m = 10$ tests and overall level $\alpha = 0.05$, at what level is each individual test conducted?
25. Explain in one or two sentences why stopping an experiment as soon as $p < 0.05$ is different from running to a pre-set n and testing once.

8 Statistical vs. practical significance

A test can hand you a tiny p-value and still be telling you nothing worth acting on. The final discipline of experimental inference is to separate three questions that beginners fuse into one: **is the effect real?** (statistical significance), **is it big enough to matter?** (practical significance), and **should it change the decision?** (decision relevance).

Definition 16 (Three kinds of significance)

- A result is **statistically significant** when the data are unlikely under H_0 — a small p-value (MAT31-2). It says the effect is probably **not zero**.
- A result is **practically significant** when the **estimated effect size** is large enough to matter for the decision at hand.
- A result has **decision relevance** when, weighing effect size against cost and risk, it actually changes what you should do.

The two can come apart in **both** directions, and recognising each case is the skill.

Worked example — significant but trivial. An A/B test on 5 million users finds that variant B lifts conversion from 20.00% to 20.05%, with $p = 0.001$ — resoundingly **statistically significant**. But the lift is 0.05 percentage points; if shipping B costs a month of engineering, it is **not practically significant** and the decision is to **not ship**. With a large enough sample, **any** non-zero effect, however negligible, becomes statistically significant.

Worked example — promising but inconclusive. A test on 300 users suggests a 5 percentage-point lift — a **practically** huge effect — but with $p = 0.12$, not **statistically significant**: the sample is too small to be sure it is real. The correct reading is not “no effect” but “promising effect, insufficient evidence” — the right decision is usually to **gather more data**, exactly the sample-size logic of Chapter 7 in reverse.

Common pitfall Never decide on the p-value alone. Report the **effect size with its confidence interval** and read the **whole** interval: if even its most optimistic end is below what would matter, a significant result is practically irrelevant; if its pessimistic end is already worth having, even a borderline-significant result may justify acting. The p-value answers “is it zero?”; the decision needs “how big, and at what cost?”

Exercises

26. Give a concrete business example, different from the text, in which a statistically significant result is **not** worth acting on.
27. A 95% confidence interval for the revenue lift of a change is $[-0.2\%, +0.4\%]$. The change costs nothing to ship. Discuss whether to ship it, referring to both ends of the interval.
28. Explain why a very large sample makes statistical significance an **unreliable** guide to whether an effect matters.

9 Likelihood geometry

The final theme returns to **estimation** — but now with two parameters at once, viewed **geometrically**. In MAT21-2 you maximised a likelihood in one parameter and read off the MLE. With **two** parameters $\theta = (\theta_1, \theta_2)$, the log-likelihood becomes a **surface** over the parameter plane, and its **shape** near the peak encodes how much the data actually pin down each parameter. We read this surface **operationally**, using the multivariable tools of MAT31-2 — gradient and Hessian — not deriving anything from scratch.

9.1 The likelihood surface

Definition 17 (Two-parameter log-likelihood) Given data and a model with parameters $\theta = (\theta_1, \theta_2)$, the **log-likelihood** $\ell(\theta)$ is the log of the probability (or density) of the observed data as a function of θ . Plotted over the (θ_1, θ_2) plane it is a **surface**; its **contour lines** (level sets) are curves of equal likelihood, and the **maximum-likelihood estimate** $\hat{\theta}$ sits at the top.

9.2 Gradient and Hessian: slope and curvature

Definition 18 (Gradient and Hessian) The **gradient**

$$\nabla \ell(\theta) = (\partial \ell / \partial \theta_1, \partial \ell / \partial \theta_2)$$

points in the steepest-uphill direction; the MLE $\hat{\theta}$ is where it **vanishes**, $\nabla \ell(\hat{\theta}) = 0$ — the flat top of the hill. The **Hessian**

$$H(\theta) = \begin{pmatrix} \partial^2 \ell / \partial \theta_1^2 & \partial^2 \ell / \partial \theta_1 \partial \theta_2 \\ \partial^2 \ell / \partial \theta_2 \partial \theta_1 & \partial^2 \ell / \partial \theta_2^2 \end{pmatrix}$$

collects the second derivatives and measures the **curvature** of the surface at the peak.

At the MLE the gradient is zero and the Hessian is **negative definite** (the surface curves downward in every direction — a peak, MAT31-1). The **magnitude** of that downward curvature is the whole story for uncertainty.

9.3 Curvature is (inverse) uncertainty

Here is the operational payoff, the reason a course on inference cares about the shape of a surface.

Proposition 19 (Curvature encodes parameter uncertainty) Near the MLE the log-likelihood is approximately a downward quadratic with Hessian H . **Sharp** curvature (large $|H|$) means a **narrow, pointed** peak and **small** uncertainty — the data determine the parameters well. **Flat** curvature means a **broad** peak and **large** uncertainty. Quantitatively, the parameter variance–covariance is approximately $(-H)^{-1}$: the **flatter** the surface, the **larger** the uncertainty.

Worked example — sharp versus flat. Two experiments estimate the same (θ_1, θ_2) . In the first, the log-likelihood falls off steeply in every direction from the peak: the contours are small tight circles, $|H|$ is large, and both parameters have small standard errors. In the second, the surface is a gentle dome: the contours are wide, $|H|$ is small, and the estimates are uncertain even though the **peak is in the same place**. The location of the MLE is identical; the **trust** you place in it is not — and only the curvature reveals the difference.

9.4 Ridges and local identifiability

The most important geometric phenomenon is the **ridge**: a surface that is sharply curved in one direction but nearly **flat** along another.

Definition 20 (Ridge and weak local identifiability) A log-likelihood **ridge** is a direction in parameter space along which ℓ is almost flat (small curvature) while being sharply curved across it. Along a ridge the data constrain a **combination** of the parameters tightly but each **individual** parameter loosely — the model is **weakly locally identifiable**. On a contour plot a ridge appears as **long, thin, tilted ellipses**: narrow across, stretched along the flat direction.

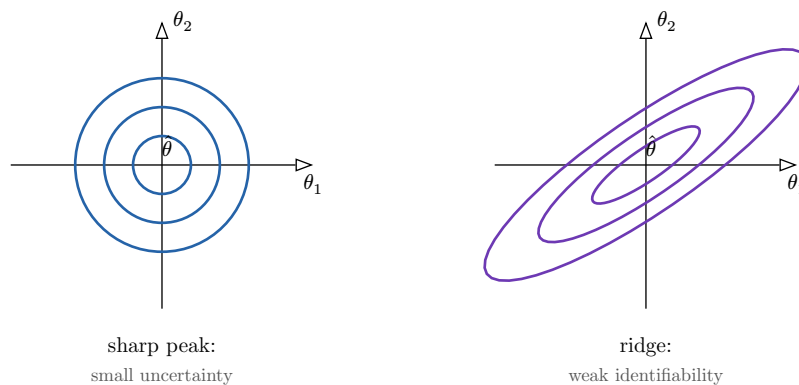


Figure 6: Contours of two-parameter log-likelihoods. **Left:** tight, round contours — sharp curvature in every direction, $|H|$ large, both parameters well-determined. **Right:** long, tilted ellipses — a **ridge**: the data fix a combination of θ_1, θ_2 (the short axis) tightly but leave the combination along the long axis nearly free, so each parameter alone is uncertain. The MLE may be the same point in both; the **curvature**, not the location, reports how much to trust it.

Common pitfall A ridge is **not** a numerical glitch — it is the data telling you that your two parameters are **entangled**: only a combination of them is well-determined. Reporting

tight standard errors for each separately would be misleading; the honest summary is the **joint** uncertainty (the tilted ellipse), which shows the parameters trade off along the ridge. Watch for ridges whenever two parameters play near-interchangeable roles in the model.

Exercises

29. At the MLE, what is the value of the gradient, and what sign-property does the Hessian have? State the geometric meaning of each.
30. Two log-likelihood surfaces have the same peak location but Hessians with $|H_1| \gg |H_2|$. Which gives more precise parameter estimates, and why?
31. Describe, in terms of contour shape, how you would **recognise** a ridge on a plotted likelihood surface, and what it implies about identifiability.
32. Explain why, on a ridge, it is more honest to report a joint confidence region than two separate standard errors.

10 A complete simulation-and-inference workflow

The chapters have built a toolkit; the OP's capstone outcome asks you to **assemble it** into a single, decision-oriented analysis. This final chapter is the integration — the checklist a competent analyst follows from question to memo.

The end-to-end workflow:

1. **State the estimand.** Write the target precisely as an expectation $\mu = \mathbb{E}[g(X)]$ or as a stochastic-system outcome $\mathbb{E}[Y]$. Be explicit about **what number the decision needs**.
2. **Choose a simulation strategy.** Direct Monte Carlo for a clean expectation; replicated system simulation for an evolving process; the bootstrap when you hold only a finite dataset and no distribution to sample from.
3. **Apply one variance-reduction technique.** Antithetic or control variates, and **report its measured effect** on the standard error and interval width — do not just claim it helped, quantify it.
4. **Compute an inferential summary.** The estimate **with a confidence interval** — a Monte Carlo CLT interval or a bootstrap percentile interval. Never a bare number.
5. **Validate.** Check against an analytical benchmark where one exists, or a convergence diagnostic where none does. State which you used.
6. **Write a decision memo.** Report the estimate and its uncertainty; separate **statistical** from **practical** significance and state the **decision**; list **limitations** (model assumptions, where the bootstrap might fail, residual Monte Carlo error) and the **validation checks** performed.

Worked example — a worked decision memo, in outline. **Question.** Should we adopt a new pricing rule? **Estimand:** expected weekly profit $\mathbb{E}[Y]$ under the new rule. **Strategy:** simulate a week of demand and pricing decisions, $n = 5000$ replications. **Variance reduction:** use last week's realised demand as a **control variate** (known mean), measured to cut the standard error by 40%. **Inference:** $\bar{Y} = \text{EUR}118\{\text{, }000$ with 95% interval $[114\{\text{, }000, 122\{\text{, }000]$. **Validation:** on a deterministic-demand special case the simulator reproduces the hand-computed profit, and the running estimate has stabilised. **Decision:** the entire interval lies above the current rule's $\text{EUR}110\{\text{, }000$, so the lift is both statistically and practically significant —

adopt, noting the limitation that the demand model is calibrated on one quarter of data and should be re-validated next quarter.

This memo is the genre the whole course has been preparing you to write: a number, its honest uncertainty, a validated method behind it, and a decision that weighs significance against what actually matters. **That** is applied probability in the service of a decision.